

ユニット 4・第5-6週

プロセスとメモリ

📅 第5-6週・日程は未定 📄 演習 TD3（解答 + 生成AIチャレンジ） 📺 アニメーション 7 本

マニュアル M4・ユニットの理論

プロセスとメモリ

ユニット 4 の基本の読み物（約 2 時間、2 週間分）。各節の最後に**実践** → リンクがあります。出力はすべて実物で、コースのマシン（Ubuntu 26.04、Linux 7.0、x86-64）で TD3 のプログラムを実行したものです。

目次

- | | | | |
|----|------------------------------|----|---|
| §1 | プロセスとそのアドレス空間 | §4 | <code>fork</code> 、コピーオンライト、バッファ |
| §2 | 仮想メモリ：ページ、フォールト、置換 | §5 | <code>exec</code> 、 <code>exit</code> 、 <code>wait</code> |
| §3 | カーネルから見たプロセス：PCB、状態、スケジューリング | §6 | 最初のプロセス、セッション、サービス |

§1 プロセスとそのアドレス空間

プロセスとは、実行中のプログラムとそのコンテキスト（メモリ、レジスタ、開いているディスクリプタ、資格情報）を合わせたものです。各プロセスは**私的なアドレス空間**を持ちます：使うアドレスは**仮想アドレス**で、2 つのプロセスで同じアドレスでも別の物理メモリを指します。x86-64 ではポインタは 64 ビットですが使うのは 48 ビット：256 TiB のうち下半分がプロセス、上半分がカーネルです。

空間は**領域**に分かれています：コード（text、読み取り専用で実行可能）、データ（初期化済みグローバル変数）と `bss`（未初期化グローバル変数）、ヒープ（小さな `malloc`）、`mmap` 領域（ライブラリと大きな `malloc`）、そしてスタック（ローカル変数、下向きに伸びる）。TD3 の `zones` プログラムは各領域のアドレスと、それを含む `/proc/self/maps` の領域を表示します：

実際の出力

```
$ ./zones
```

zone		address	perm	region
code	(main)	0x5f935f9d1350	r-xp	~/fso/tds/bin/td3/zones
data	(initialized)	0x5f935f9d5010	rw-p	~/fso/tds/bin/td3/zones
bss	(global_array)	0x5f935f9d5040	rw-p	~/fso/tds/bin/td3/zones
heap	(malloc 10 B)	0x5f938dc96010	rw-p	[heap]
heap	(end: sbrk(0))	0x5f938dcb6fff	rw-p	[heap]
mmap	(malloc 64 MiB)	0x71b340fff010	rw-p	(anonymous)
libc	(printf)	0x71b345064080	r-xp	/usr/lib/x86_64-linux-gnu/libc.so.6
stack	(local)	0x7fff6c1e68b4	rw-p	[stack]

何度か実行するとアドレスが変わります：攻撃者がコードやスタックの位置を予測できないよう、カーネルが各領域の基点をランダム化しているのです（**ASLR**、アドレス空間配置のランダム化）。glibc は 128 KiB 以上の `malloc` にはヒープの外に専用の `mmap` を使い、解放されるとすぐにシステムに返します。

実践 → TD3・5（zones）

§2 仮想メモリ：ページ、フォールト、置換

メモリは**ページ**（x86-64 では 4 KiB。2 MiB と 1 GiB のヒューズページもある）に、物理メモリは同じ大きさの**フレーム**に分けられます。プロセスごとの**ページテーブル**が、各ページがどのフレームにあり、どんな権限（読み、書き、実行、ユーザー）を持つかを示します。x86-64 では表は 4 段で、仮想アドレスは 9 ビットの添字 4 つと 12 ビットのオフセットに分けられます。ハードウェア（**MMU**）がアクセスのたびに交換し、**TLB** が最近の交換をキャッシュして毎回表をたどらずに済むようにします。

表のエントリが存在しなければ、MMU は**ページフォールト**を起こしてカーネルに飛びます。アドレスが有効な領域に属していれば、カーネルはフレームを割り当ててゼロで埋め（またはディスクから読み）、命令をやり直します：これが**遅延割り当て**です。どの領域にも属さない場合や、アクセスが権限に反する場合（コードへの書き込み、スタックでの実行）は `SIGSEGV` を送ります。lazy で測った結果：

実際の出力

```
$ ./lazy 1024
start                                VmRSS =      1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =      1952 KiB    page faults so far = 8
after touching half of the pages     VmRSS =    526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each) page faults: 131073    1968 ns per fault
after free                            VmRSS =      2012 KiB    page faults so far = 131081
```

Linux は**楽観的**でもあります（オーバーコミット）：ほとんど誰も予約したすべてを使わないので、持っている以上のメモリを約束します。本当に足りなくなると **OOM キラー**がプロセスを終了させます。空きフレームがなくなると、カーネルは**犠牲**になるページを選び、ディスクに送る（スワップ）か RAM 上で圧縮します（zswap、zram）。理想的なアルゴリズム（OPT）は最も遅く使われるページを追い出しますが、実際のアルゴリズムは MMU が立てるアクセスビットを使って LRU を近似します（クロックアルゴリズム。Linux ではアクティブ/インアクティブのリストと MGLRU）。FIFO では **Belady の異常**が起こりえます：フレームを増やすとフォールトも増えるのです。使用中のページの集合がメモリに収まらないと、システムは実行よりページングに時間を使うようになります：**スラッシング**です。

実践 → A4.1 仮想メモリ・A4.5 ページ置換

§3 カーネルから見たプロセス：PCB、状態、スケジューリング

カーネルは各プロセスを**制御ブロック**（PCB。Linux では `struct task_struct`）で表します：PID と PPID、所有者と権限、状態、保存したレジスタ、優先度、CPU 時間、ディスクリプタ表、制限

(`getrlimit`)、シグナル、ページテーブル。`/proc/PID/` で調べられ、`ps` や `top` はそれを読んでいきます。

プロセスはいくつかの**状態**を経ます：**実行可能**（CPU 待ち）と**実行中**（どちらも `ps` では `R`）、入出力待ちでブロック（`S`、割り込めなければ `D`）、`SIGSTOP` / `SIGTSTP` で**停止**（`T`）、そして終了から親の `wait` までの**ゾンビ**（`Z`）。

CPU をあるプロセスから別のプロセスへ渡すのが**コンテキストスイッチ**です：一方のレジスタを保存し、他方のレジスタを読み込み、ページテーブルを切り替えます。誰の番かは**スケジューラ**が決めます。古典的なアルゴリズム：FCFS（到着順。コンボイ効果に悩まされる）、SJF と SRTF（短いもの優先。平均待ち時間は最小だが長さの見積もりが必要）、ラウンドロビン（クォンタムずつの順番。応答はよいがコンテキストスイッチが増える）、優先度（飢餓の危険があり、エージングで対処）。バージョン 6.6（2023 年）以降、Linux は **EEVDF** を使います：優先度（`nice`）で重み付けした仮想時間で CPU を分け、最も早い締め切りを先に処理します。

実践 → A4.3 状態・A4.4 スケジューリング

§4 `fork`、コピーオンライト、バッファ

Unix では新しいプロセスは別のプロセスの**複製**として生まれます：`fork()` は子では 0、親では子の PID、失敗時は -1 を返します。子はすべてのコピーを受け取ります：メモリ、ディスクリプタ

（オープンファイル表の同じエントリを指す、ユニット 3）、シグナルハンドラとマスク（保留中のシグナルは除く）。全メモリのコピーは高くつくので、Linux は**コピーオンライト**を使います：ページテーブルだけをコピーしてすべてを読み取り専用にし、どちらかが書き込んだページだけをコピーします：

実際の出力

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per copy
parent: the child exited with status 0; my copy still says 'x'
```

コピーには C ライブラリが保留しているものも含まれます。`printf` は `write` を呼ばずに**バッファ**にためます。バッファは 1 行ごと（端末への `stdout`）、満杯になったとき（パイプやファイルへの `stdout`）、`fflush` で、または `exit` 時にフラッシュされます。`stderr` はバッファされず、`_exit` は何もフラッシュしません。`fork` の時点でバッファにテキストが残っていれば、親と子の両方がそれを書くので 2 回表示されます。

実際の出力

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

実践 → A4.2 コピーオンライト・A4.6 バッファ・TD3・2 と 3

§5 exec、exit、wait

`execve(path, argv, envp)` はプロセスのプログラムを置き換えます：新しいコードとデータを読み込み、新しいヒープとスタックを作り、エントリポイントへジャンプします。PID、PPID、開いているディスクリプタ（`O_CLOEXEC` 付きのものを除く）は残ります。SUID ビットがあれば実効 UID がファイルの所有者のものになります。成功すると戻りません。ライブラリの派生版（`execl`、`execv`、`execvp` ...）は、引数の渡し方や `PATH` を探すかどうかが違います。`fork` と `exec` の間に子はディスクリプタを組み替えられます：シェルはこうして `>`、`<`、パイプを実装しています（TD3・4 の `launch` とユニット 5）。

プロセスは `exit(status)`（`atexit` の関数を呼び、`stdio` のバッファをフラッシュし、システムコール `_exit` を呼ぶ）か、シグナルで終了します。カーネルはファイルを閉じ、メモリを解放し、生きている子を `init` か最も近い `subreaper` に引き渡し、`SIGCHLD` で親に知らせます。PCB は親が `wait` か `waitpid` を呼ぶまでゾンビとして残り、それらは詰め込まれた終了状態を返します。マクロで解説します：

`WIFEXITED` / `WEXITSTATUS`、`WIFSIGNALED` / `WTERMSIG`、`WIFSTOPPED`、`WIFCONTINUED`（`WUNTRACED` と `WCONTINUED` のオプション付き。`WNOHANG` はブロックしない）。

実際の出力

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
   grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

実際の出力

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

実践 → A4.7 fork、exec と wait・TD3・4

§6 最初のプロセス、セッション、サービス

すべてのプロセスが `fork` から生まれるなら、誰かが最初でなければなりません。**プロセス 0** は起動時にカーネルが作ります (CPU ごとに1つ、カーネルモード) : 仮想メモリを初期化し、カーネルスレッド (`kthreadd`、PID 2) と **プロセス 1** を作ります。プロセス 1 はユーザーモードに切り替えて `/sbin/init` (現在はほとんどのディストリビューションで **systemd**) を実行し、サービスを起動し、孤児を引き取ります。コースのマシンでは `ps -p 1 -o comm=` が `systemd` と答えます (TDO)。

セッション はリーダーを持つプロセスの集まりで、ふつうは1つの端末のものです。リーダーが終了すると他のプロセスは `SIGHUP` を受け取ります。 `setsid()` は制御端末のない新しいセッションを作ります。古典的な **デーモン** は `fork` で端末から離れ、親は終了し、子が `setsid` を呼びました。現在の Linux ではサービスは `systemd` が起動します : プログラムはフォアグラウンドで動き、セッション、ログ (`journalctl`)、再起動、資源の制限 (`cgroups`、ユニット 9) は `systemd` が引き受けます。